

Introduction to EDA

EDA (Exploratory Data Analysis) is the process of examining, summarizing, and visualizing a dataset to understand its structure, identify patterns, and find problems in the data before applying Machine Learning models.

a) Importance of EDA in the ML Workflow

EDA is an important step in the Machine Learning workflow because it helps us understand the data before training a model.

1. Understand the Dataset

EDA helps us understand:

- Number of rows and columns
- Data types
- Numerical and categorical features
- Basic statistics of the data

```
df.shape  
df.info()  
df.head()  
df.describe()
```

2. Identify Missing Values

EDA helps us find missing values in the dataset.

```
df.isnull().sum()
```

Missing values can then be handled using methods such as deletion or imputation.

3. Detect Outliers

EDA helps identify unusually high or low values.

For example:

Age: 21, 22, 23, 24, 25, 150

Here, 150 may be an outlier.

Common methods for detecting outliers include:

- Z-score
- IQR
- Box plot

4. Understand Data Distribution

EDA helps us understand how the values of a feature are distributed.

Histograms and density plots can help identify:

- Normal distribution
- Skewness
- Extreme values
- Different groups in the data

5. Find Relationships Between Features

EDA helps us identify relationships between variables.

For example, experience and income may have a positive relationship.

```
df.corr(numeric_only=True)
```

6. Identify Incorrect or Inconsistent Data

EDA can reveal incorrect values and inconsistencies.

Examples:

Age = -5

Gender = Male, male, M

Salary = 50000, "50000 INR"

These issues should be corrected before model training.

7. Helps in Data Preprocessing

EDA helps us decide which preprocessing techniques are required, such as:

- Missing value imputation
- Outlier handling
- Encoding
- Scaling
- Transformation
- Feature selection
- Feature engineering

8. Helps Build Better ML Models

By understanding and cleaning the dataset, we can provide better-quality data to the Machine Learning model, which can help improve the reliability of the model.

b) Overview of EDA Steps

The main steps of EDA are:

Dataset



Understand the Data



Check Data Structure



Check Data Quality



Univariate Analysis



Bivariate Analysis



Multivariate Analysis



Identify Patterns and Problems



Preprocess the Data



Machine Learning Model

Step 1: Load the Dataset

```
import pandas as pd
```

```
df = pd.read_csv("data.csv")
```

Step 2: Understand the Data

```
df.head()  
df.tail()  
df.shape  
df.info()  
df.describe()
```

These functions help us understand the structure and basic statistics of the dataset.

Step 3: Check Data Quality

Missing Values

```
df.isnull().sum()
```

Duplicate Values

```
df.duplicated().sum()
```

Unique Values

```
df.nunique()
```

Check Value Range

```
df["age"].min()  
df["age"].max()
```

Step 4: Univariate Analysis

Univariate analysis means analyzing **one variable at a time**.

Example:

```
df["age"].hist()
```

Common techniques:

- Mean
- Median
- Mode
- Histogram
- Box plot
- Count plot

Step 5: Bivariate Analysis

Bivariate analysis means analyzing **two variables at a time**.

Example:

Experience → Income

It helps us understand whether two variables have a relationship.

Common techniques:

- Scatter plot
- Correlation
- Box plot
- Grouped analysis

Step 6: Multivariate Analysis

Multivariate analysis means analyzing **three or more variables together**.

Common techniques:

- Correlation heatmap
- Pair plot
- Multivariable visualizations

Step 7: Identify Patterns and Problems

During EDA, we look for:

- Missing values
- Outliers
- Skewed distributions

- Incorrect values
- Duplicate records
- Strong correlations
- Categorical inconsistencies

Step 8: Prepare Data for Machine Learning

Based on the findings from EDA, we perform appropriate preprocessing.

Missing values → Imputation
 Outliers → Outlier handling
 Categorical data → Encoding
 Skewed data → Transformation
 Different scales → Scaling
 Irrelevant features → Feature selection
 Useful combinations → Feature engineering

After these steps, the dataset can be used for **Machine Learning model training**.

Quick Revision

EDA = Explore → Understand → Detect → Prepare

- **Univariate** → One variable
- **Bivariate** → Two variables
- **Multivariate** → Three or more variables
- **EDA before ML** → Understand and identify problems in the data
- **Main goal** → Prepare better-quality data for Machine Learning

One-line Definition

EDA is the process of exploring, summarizing, and visualizing data to understand its structure, identify patterns and problems, and prepare it for Machine Learning.

Data Visualization with Matplotlib

a) Introduction to Matplotlib

Matplotlib is a popular Python library used for **data visualization**. It helps us represent data graphically using different types of charts and plots.

It is commonly used during **EDA (Exploratory Data Analysis)** to understand patterns, trends, distributions, and relationships in data.

Importing Matplotlib

```
import matplotlib.pyplot as plt
```

Here, **pyplot** provides functions for creating and customizing plots.

b) Basic Plots

1. Line Plot

A **line plot** is used to show **trends or changes over a continuous or ordered variable**.

Example

```
x = [1, 2, 3, 4, 5]
y = [10, 20, 15, 30, 25]
```

```
plt.plot(x, y)
plt.show()
```

Common uses

- Time-series data
- Trends
- Changes over time
- Comparing continuous values

Example:

```
Month → Sales
Jan  → 100
Feb  → 150
Mar  → 130
Apr  → 180
```

2. Bar Plot

A **bar plot** is used to compare values between different **categories**.

Example

```
categories = ["A", "B", "C", "D"]  
values = [20, 35, 25, 40]
```

```
plt.bar(categories, values)  
plt.show()
```

Common uses

- Comparing categories
 - Comparing sales of products
 - Comparing student marks
 - Showing counts
-

3. Histogram

A **histogram** is used to show the **distribution of numerical data**.

Example

```
ages = [18, 19, 20, 21, 21, 22, 23, 24, 25, 25, 26, 27]
```

```
plt.hist(ages)  
plt.show()
```

A histogram divides numerical values into **bins** and shows how many observations fall into each range.

Common uses

- Understanding data distribution
- Detecting skewness
- Identifying possible outliers

- Understanding frequency
-

c) Customizing Plots

Matplotlib allows us to customize plots to make them easier to understand.

1. Adding Labels

Use:

```
plt.xlabel("X-axis")  
plt.ylabel("Y-axis")
```

Example:

```
plt.plot(x, y)  
  
plt.xlabel("Days")  
plt.ylabel("Sales")  
  
plt.show()
```

2. Adding a Title

Use:

```
plt.title("Sales Trend")
```

Example:

```
plt.plot(x, y)  
  
plt.title("Sales Trend")  
plt.xlabel("Days")  
plt.ylabel("Sales")  
  
plt.show()
```

3. Adding a Legend

A **legend** identifies different data series in a plot.

```
plt.plot(x, y, label="Sales")  
plt.legend()
```

```
plt.show()
```

For multiple lines:

```
x = [1, 2, 3, 4, 5]
```

```
sales = [10, 20, 15, 30, 25]
```

```
profit = [5, 10, 8, 15, 12]
```

```
plt.plot(x, sales, label="Sales")  
plt.plot(x, profit, label="Profit")
```

```
plt.xlabel("Days")  
plt.ylabel("Amount")  
plt.title("Sales and Profit")  
plt.legend()
```

```
plt.show()
```

4. Changing Colors

We can change the color of a plot using the `color` parameter.

```
plt.plot(x, y, color="red")  
plt.show()
```

For a bar plot:

```
plt.bar(categories, values, color="green")  
plt.show()
```

Some commonly used colors are:

red
blue
green
black
orange
purple

Complete Example

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]
sales = [10, 20, 15, 30, 25]

plt.plot(x, sales, color="blue", label="Sales")

plt.title("Sales Trend")
plt.xlabel("Days")
plt.ylabel("Sales")
plt.legend()

plt.show()
```

Important Functions

Function	Purpose
<code>plt.plot()</code>	Line plot
<code>plt.bar()</code>	Bar plot
<code>plt.hist()</code>	Histogram

```
plt.xlabel    X-axis label
l()

plt.ylabel    Y-axis label
l()

plt.title     Plot title
()

plt.legend    Display legend
d()

plt.show(     Display plot
)
```

Quick Revision

Matplotlib → Visualization library

- **Line plot** → Trends
- **Bar plot** → Category comparison
- **Histogram** → Distribution
- **xlabel/ylabel** → Axis labels
- **title** → Plot title
- **legend** → Identifies data series
- **color** → Changes plot color
- **show()** → Displays the plot

Data Visualization with Seaborn

a) Introduction to Seaborn

Seaborn is a Python data visualization library built on top of **Matplotlib**. It is mainly used for creating attractive and informative **statistical visualizations**.

Seaborn works well with **Pandas DataFrames** and makes it easier to create complex statistical plots.

Importing Seaborn

```
import seaborn as sns
import matplotlib.pyplot as plt
```

Matplotlib vs Seaborn

- **Matplotlib** → General-purpose visualization with more low-level control.
 - **Seaborn** → Statistical visualization with simpler syntax and better default styling.
-

b) Advanced Plots

1. Box Plot

A **box plot** is used to understand the **distribution of numerical data** and identify **outliers**.

A box plot represents:

- Minimum
- First Quartile (Q1)
- Median
- Third Quartile (Q3)
- Maximum
- Outliers

Example

```
sns.boxplot(x=df["income"])  
plt.show()
```

Uses

- Detecting outliers
 - Understanding data spread
 - Comparing distributions
 - Identifying the median and quartiles
-

2. Violin Plot

A **violin plot** combines the features of a **box plot** and a **density plot**.

It shows:

- Median and quartiles
- Spread of data
- Shape of the distribution
- Density of values

Example

```
sns.violinplot(x=df["income"])  
plt.show()
```

Box Plot vs Violin Plot

Box plot:

- Shows summary statistics
- Makes outliers easy to identify

Violin plot:

- Shows summary statistics
 - Also shows the **shape and density of the distribution**
-

3. Pair Plot

A **pair plot** is used to visualize relationships between **multiple numerical features** in a dataset.

Example

```
sns.pairplot(df)  
plt.show()
```

A pair plot contains:

- Scatter plots between different features
- Distribution plots along the diagonal

For example, if a dataset contains:

Age
Income

Experience
Education

A pair plot allows us to examine relationships between these variables simultaneously.

Uses

- Finding relationships between features
 - Identifying patterns
 - Identifying clusters
 - Understanding distributions
-

c) Customizing Seaborn Plots

Seaborn plots can be customized using different parameters and Matplotlib functions.

1. Adding a Title

```
plt.title("Income Distribution")
```

2. Adding Axis Labels

```
plt.xlabel("Age")  
plt.ylabel("Income")
```

3. Changing the Color

```
sns.boxplot(x=df["income"], color="skyblue")  
plt.show()
```

The `color` parameter changes the color of the plot.

4. Using Color Palettes

A **color palette** controls the colors used in a Seaborn plot.

```
sns.boxplot(  
    x="education",  
    y="income",  
    data=df,  
    palette="Set2"  
)  
plt.show()
```

Common palettes include:

```
"Set1"  
"Set2"  
"Set3"  
"coolwarm"  
"viridis"
```

5. Changing the Plot Style

Seaborn provides different styles for changing the overall appearance of plots.

```
sns.set_style("whitegrid")
```

Common styles:

```
"darkgrid"  
"whitegrid"  
"dark"  
"white"  
"ticks"
```

Complete Example

```
import seaborn as sns
```

```
import matplotlib.pyplot as plt
```

```
sns.set_style("whitegrid")
```

```
sns.boxplot(  
    x="education",  
    y="income",  
    data=df,  
    palette="Set2"  
)
```

```
plt.title("Income by Education")  
plt.xlabel("Education")  
plt.ylabel("Income")
```

```
plt.show()
```

Quick Revision

Plot / Function	Purpose
<code>sns.boxplot()</code>	Distribution + outliers
<code>sns.violinplot()</code>	Distribution + density
<code>sns.pairplot()</code>	Relationships between multiple features
<code>color</code>	Changes plot color
<code>palette</code>	Controls color scheme
<code>sns.set_style()</code>	Changes overall plot style
<code>plt.title()</code>	Adds title
<code>plt.xlabel()</code>	X-axis label
<code>plt.ylabel()</code>	Y-axis label

Remember

Seaborn → Statistical Data Visualization

- **Box plot** → Distribution + outliers
- **Violin plot** → Distribution + density shape
- **Pair plot** → Relationships between multiple features
- **Palette** → Color scheme
- **Set style** → Overall appearance

Interpreting EDA Results

After performing EDA, the next step is to **interpret the data** and communicate what we discovered.

a) Identifying Patterns and Trends in Data

A **pattern** is a repeated relationship or behavior found in the data.

A **trend** is the general direction in which a value changes over time or across observations.

Examples

- Sales increasing every month → **Increasing trend**
- Sales decreasing every month → **Decreasing trend**
- Higher experience associated with higher income → **Positive relationship**
- Most customers belonging to one age group → **Pattern**
- A sudden unusual value → **Possible outlier**

Patterns and trends can be identified using:

- Line plots
- Bar plots
- Histograms
- Scatter plots
- Box plots
- Correlation analysis

The main goal is to understand **what is happening in the data and identify meaningful relationships or changes**.

b) Drawing Actionable Insights from Data

An **insight** is a meaningful conclusion obtained from analyzing data.

An **actionable insight** is a finding that can help someone decide **what action to take**.

Example

Suppose a company's data shows:

Customers aged 18–25 → 70% of online purchases

Finding:

Young customers account for a large portion of online purchases.

Actionable insight:

The company could investigate targeted products or marketing campaigns for this age group.

Another Example

Observation:

Income has a strong positive relationship with years of experience.

Insight:

More experienced employees tend to have higher income in this dataset.

Important:

Correlation does not automatically mean causation. A relationship between two variables does not prove that one variable causes the other.

c) Communicating Findings Effectively

Data analysis is useful only when the results can be **clearly communicated** to others.

Effective communication should include:

1. State the Finding

Clearly explain what you discovered.

Example:

Income is positively associated with years of experience in this dataset.

2. Support the Finding with Evidence

Use:

- Charts
- Tables
- Summary statistics
- Correlation values
- Relevant measurements

3. Explain the Meaning

Do not simply show a graph. Explain **what the graph tells us**.

4. Keep Visualizations Simple

A good visualization should:

- Have a clear title
- Have meaningful axis labels
- Use an appropriate chart type
- Avoid unnecessary elements
- Highlight important information

5. Mention Limitations

Do not make conclusions that the available data cannot support.

Example:

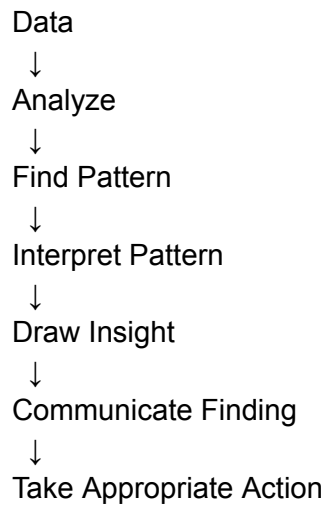
Experience and income are correlated, but this analysis alone does not prove that experience causes higher income.

Quick Revision

EDA Interpretation = Pattern → Insight → Action → Communication

- **Pattern** → Something repeatedly observed in the data.
- **Trend** → General direction of change.
- **Insight** → Meaningful conclusion from the data.
- **Actionable Insight** → Finding that can inform a practical action or decision.
- **Communication** → Presenting findings clearly using explanations, charts, and evidence.

Simple Flow



Correlation Analysis

a) Understanding Correlation and Its Importance

Correlation measures the **strength and direction of the relationship between two variables**.

For example, if **experience_years** increases and **income** also tends to increase, the two variables have a **positive correlation**.

Types of Correlation

1. Positive Correlation

Both variables tend to increase or decrease together.

Example:

Experience ↑ → **Income** ↑

2. Negative Correlation

When one variable increases, the other tends to decrease.

Example:

Price ↑ → **Demand** ↓

3. No Correlation

There is no clear linear relationship between the variables.

Correlation Coefficient

The correlation coefficient ranges from **-1 to +1**.

Value	Meaning
+1	Perfect positive correlation
+0.7	Strong positive correlation
+0.3	Weak positive correlation
0	No linear correlation
-0.3	Weak negative correlation
-0.7	Strong negative correlation
-1	Perfect negative correlation

Importance of Correlation Analysis

Correlation helps us to:

- Understand relationships between variables.
- Identify important features.
- Detect highly related or redundant features.
- Perform feature selection.
- Understand the dataset before building an ML model.
- Detect possible multicollinearity between features.

Important: Correlation does not prove causation.

For example, if `experience_years` and `income` are correlated, it does not automatically mean that experience alone causes higher income.

b) Calculating Correlation Coefficients

There are two commonly used correlation methods:

1. **Pearson Correlation**

2. Spearman Correlation

1. Pearson Correlation

Pearson correlation measures the **strength and direction of a linear relationship** between two numerical variables.

Formula

$$r = \frac{\text{Cov}(X, Y)}{\sigma_X \sigma_Y} = \frac{\text{Cov}(X, Y)}{\sigma_X \sigma_Y}$$

Where:

- r = Pearson correlation coefficient
- $\text{Cov}(X, Y)$ = covariance between X and Y
- σ_X = standard deviation of X
- σ_Y = standard deviation of Y

Python

```
df["experience_years"].corr(  
    df["income"],  
    method="pearson"  
)
```

To calculate Pearson correlation for all numerical columns:

```
df.corr(method="pearson", numeric_only=True)
```

If the result is:

0.85

It indicates a **strong positive linear relationship** between the two variables.

2. Spearman Correlation

Spearman correlation measures the relationship between variables based on their **ranks**.

It is useful when:

- The relationship is monotonic but not necessarily linear.
- The data contains outliers.
- The variables are ordinal or ranked.
- Pearson correlation is not suitable.

Python

```
df["experience_years"].corr(  
    df["income"],  
    method="spearman"  
)
```

For all numerical columns:

```
df.corr(method="spearman", numeric_only=True)
```

Pearson vs Spearman

Pearson	Spearman
Measures linear relationship	Measures monotonic relationship
Uses actual values	Uses ranks
More sensitive to outliers	Generally less sensitive to outliers
Suitable for linear relationships	Suitable for monotonic relationships

Easy Way to Remember

Pearson → Linear relationship

Spearman → Rank/monotonic relationship

c) Visualizing Correlations Using Heatmaps

A **correlation heatmap** is a visual representation of correlation coefficients between multiple variables.

It helps us identify:

- Strong positive relationships
- Strong negative relationships
- Weak relationships
- Highly correlated features

Using Seaborn

```
import seaborn as sns  
import matplotlib.pyplot as plt
```

```
corr = df.corr(numeric_only=True)
```

```
sns.heatmap(  
    corr,  
    annot=True,  
    cmap="coolwarm"  
)
```

```
plt.title("Correlation Heatmap")  
plt.show()
```

Understanding the Heatmap

Suppose the heatmap shows:

experience_years ↔ income = 0.85

This indicates a **strong positive correlation**.

If it shows:

age ↔ job_satisfaction = -0.10

This indicates a **very weak negative correlation**.

Important Parameters

```
sns.heatmap(  
    corr,  
    annot=True,  
    cmap="coolwarm",  
    fmt=".2f"  
)
```

- `corr` → correlation matrix
- `annot=True` → displays correlation values
- `cmap` → controls the color scheme
- `fmt=".2f"` → displays values up to 2 decimal places

Complete Practical Example

```
import pandas as pd
```

```
import seaborn as sns
import matplotlib.pyplot as plt

# Calculate Pearson correlation
pearson_corr = df.corr(
    method="pearson",
    numeric_only=True
)

# Display correlation matrix
display(pearson_corr)

# Visualize correlation matrix
plt.figure(figsize=(10, 6))

sns.heatmap(
    pearson_corr,
    annot=True,
    cmap="coolwarm",
    fmt=".2f"
)

plt.title("Pearson Correlation Heatmap")
plt.show()
```

Quick Revision

Correlation → Relationship between two variables

Pearson → Measures linear relationship

Spearman → Measures rank/monotonic relationship

Range → -1 to +1

+1 → Perfect positive correlation

0 → No linear correlation

-1 → Perfect negative correlation

Heatmap → Visual representation of correlations

Remember: Correlation ≠ Causation.

Covariance Analysis

a) Understanding Covariance and Its Importance

Covariance measures how two variables **change together**.

It tells us whether two variables tend to increase or decrease together.

Types of Covariance

1. Positive Covariance

When one variable increases, the other variable also tends to increase.

Example:

Experience $\uparrow \rightarrow$ **Income** $\uparrow$

Therefore, covariance is **positive**.

2. Negative Covariance

When one variable increases, the other variable tends to decrease.

Example:

Price $\uparrow \rightarrow$ **Demand** $\downarrow$

Therefore, covariance is **negative**.

3. Covariance Close to Zero

When covariance is close to zero, there is little or no **linear relationship** between the two variables.

Importance of Covariance

Covariance is useful for:

- Understanding how variables change together.
- Studying relationships between variables.
- Calculating correlation.
- Understanding relationships between features in an ML dataset.
- Creating a covariance matrix.
- Statistical and machine learning analysis.

Covariance vs Correlation

Covariance	Correlation
Shows the direction of the relationship	Shows direction and strength
Has no fixed range	Always ranges from -1 to +1
Depends on the scale and units of variables	Unit-free
Difficult to compare between datasets	Easier to interpret and compare

Easy way to remember:

Covariance → How variables change together
Correlation → How strongly variables are related

b) Calculating Covariance

Formula

Sample covariance is calculated using:

$$\text{Cov}(X, Y) = \frac{\sum_{i=1}^n (X_i - \bar{X})(Y_i - \bar{Y})}{n-1}$$

Where:

- X_i = individual value of X
- Y_i = individual value of Y
- $\bar{X}$ = mean of X
- $\bar{Y}$ = mean of Y

- n = number of observations

Using Pandas

To calculate covariance between two columns:

```
df["experience_years"].cov(df["income"])
```

To calculate the covariance matrix:

```
df.cov(numeric_only=True)
```

The covariance matrix contains the covariance between every pair of numerical variables.

The values on the **diagonal** represent the covariance of a variable with itself, which is equal to its **variance**.

For example:

	age	income
age	25	8500
income	8500	450000

Here:

- Age ↔ Income = 8500 → Positive covariance
 - Age ↔ Age = 25 → Variance of Age
 - Income ↔ Income = 450000 → Variance of Income
-

c) Visualizing Covariance Matrices

A **covariance matrix** shows the covariance between every pair of numerical variables.

We can first calculate it using Pandas:

```
cov_matrix = df.cov(numeric_only=True)
```

```
display(cov_matrix)
```

We can visualize the covariance matrix using a **heatmap**.

```
import seaborn as sns
```

```
import matplotlib.pyplot as plt
```

```
cov_matrix = df.cov(numeric_only=True)
```

```
plt.figure(figsize=(10, 6))
```

```
sns.heatmap(  
    cov_matrix,  
    annot=True,  
    cmap="coolwarm",  
    fmt=".2f"  
)
```

```
plt.title("Covariance Matrix")
```

```
plt.show()
```

Important Parameters

- `cov_matrix` → covariance matrix to visualize
- `annot=True` → displays covariance values inside the heatmap
- `cmap="coolwarm"` → sets the color scheme
- `fmt=".2f"` → displays values up to 2 decimal places
- `plt.figure(figsize=(10, 6))` → sets the size of the figure

Understanding the Covariance Matrix

The covariance matrix is **symmetric**.

For example:

$\text{Cov}(\text{Age}, \text{Income}) = \text{Cov}(\text{Income}, \text{Age})$

The diagonal contains the **variance** of each variable.

Covariance Matrix vs Correlation Matrix

Covariance Matrix

```
cov_matrix = df.cov(numeric_only=True)
```

Shows **how variables change together**.

Correlation Matrix

```
corr_matrix = df.corr(numeric_only=True)
```

Shows the **strength and direction of the relationship**, with values between **-1 and +1**.

Quick Revision

Covariance → Measures how two variables change together.

Positive covariance → Variables tend to increase together.

Negative covariance → One tends to increase while the other decreases.

Zero/near-zero covariance → Little linear relationship.

Covariance matrix → Covariance between all numerical variables.

Diagonal of covariance matrix → Variance of each variable.

Heatmap → Visual representation of the covariance matrix.